Wie aus Rohdaten zwei Tabellen werden

Der Ordner prep/ ist die Küche des Projekts. Hier kommen die Rohdaten von Feuerwehr, Polizei, Statistikamt und Planungsamt an, und heraus gehen zwei fertige Tabellen, mit denen jeder spätere Schritt rechnet. Dieses Heft zeigt, welche Datei dabei welche Aufgabe hat und wofür jede einzelne Funktion da ist. Dazu kommt die Prüfdatei aus tests/, die das Ergebnis nachkontrolliert.

Ordner: prep/ und tests/
Umfang: 8 Dateien, 71 Funktionen
Anschluss an: Band 1 und Band 2 der Laienfassung. Fachbegriffe von dort werden vorausgesetzt, neue sind beim ersten Auftreten erklärt.
Grundlage: der Quelltext im Abgabeordner, Stand 08.10.2026

Heft 1
prep/ und tests/

Heft 2
vorpruefung/

Heft 3
modelle/

TEIL 1

Der Ordner auf einen Blick

> Die Aufbereitung ist in zwei Schritte geteilt, die nacheinander laufen. Alles andere in diesem Ordner hilft diesen beiden Schritten, beschreibt ihr Ergebnis oder prüft es.

Schritt 1 steckt in `s1_daten.py`. Er holt die Rohdaten und hängt an jeden einzelnen Feuerwehreinsatz die Merkmale seines Stadtteils: Sozialdaten, Kriminalitätsbelastung, Bebauung. Schritt 2 steckt in `s2_datensaetze.py`. Er nimmt aus den rund 720.000 Einsätzen die der 36 Stadtteile im Analysezeitraum, verdichtet sie zu einer Zeile je Stadtteil und Monat und schreibt hinein, welcher Stadtteil in welcher Prüfgruppe steht. `build.py` startet beide Schritte mit einem einzigen Befehl, `config.py` hält alle Festlegungen, auf die sich die beiden stützen.

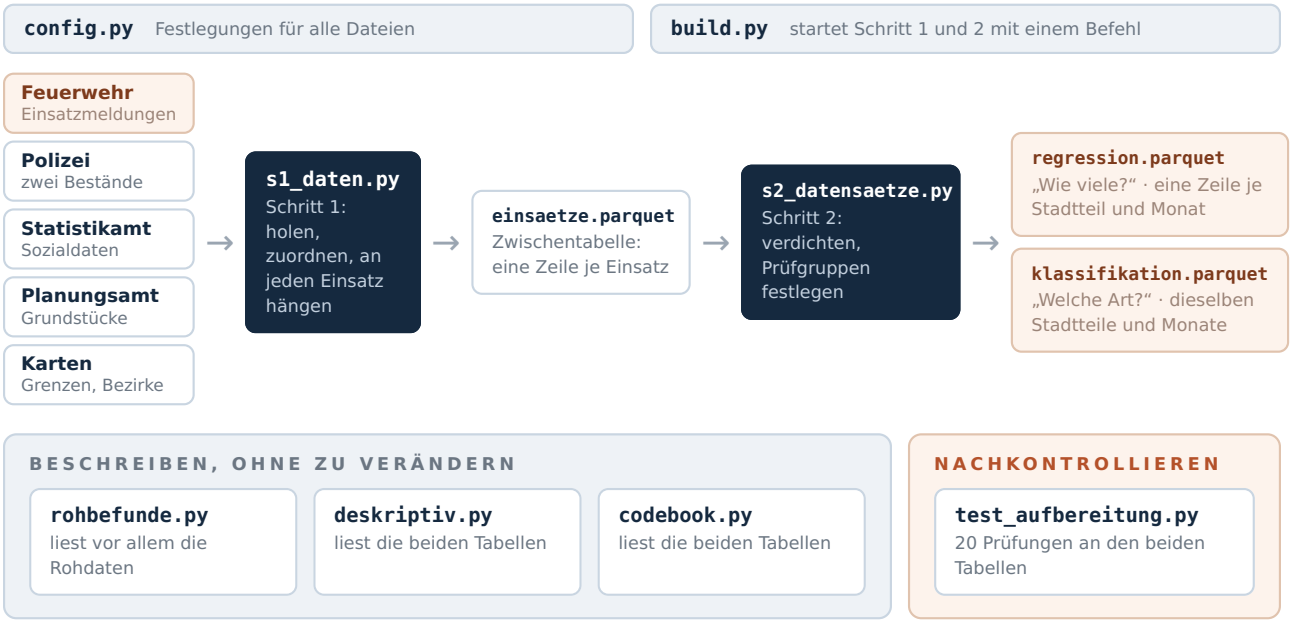


Abbildung 1: Der Weg der Daten durch den Ordner. Nur `s1_daten.py` und `s2_datensaetze.py` erzeugen Daten. Die drei Dateien in der linken unteren Leiste lesen nur und legen ihre Befunde in `results/` ab. Die Prüfdatei aus `tests/` kontrolliert die beiden fertigen Tabellen.

DATEI	AUFGABE IN EINEM SATZ	FUNKTIONEN
`config.py`	Hält alle Festlegungen der Aufbereitung an einer Stelle.	0
`build.py`	Startet Schritt 1 und Schritt 2 mit einem Befehl.	3
`s1_daten.py`	Schritt 1: Rohdaten holen und an jeden Einsatz hängen.	15
`s2_datensaetze.py`	Schritt 2: zu Stadtteil-Monaten verdichten und die Prüfgruppen festlegen.	10
`rohbefunde.py`	Beschreibt die Rohdaten so, wie sie vom Portal kommen.	2
`deskriptiv.py`	Beschreibt, wie die fertige Tabelle aussieht.	10
`codebook.py`	Erklärt, was jede Spalte bedeutet.	8
`tests/test_aufbereitung.py`	Prüft die beiden fertigen Tabellen mit 20 Prüfungen.	23

TEIL 2

Wie sich die Dateien voneinander abgrenzen

> Acht Dateien sind viel für eine Aufgabe, die man in einem Satz sagen kann. Die Aufteilung folgt aber einer einfachen Regel: Jede Datei beantwortet genau eine Frage.

Verändern oder nur ansehen

Die wichtigste Grenze verläuft zwischen den Dateien, die Daten erzeugen, und denen, die sie nur ansehen. `s1_daten.py` und `s2_datensaetze.py` schreiben die Dateien, mit denen später gerechnet wird. `rohbefunde.py`, `deskriptiv.py` und `codebook.py` lesen nur und legen ihre Befunde in `results/` ab. Man kann sie beliebig oft laufen lassen, ohne dass sich an den Analysedaten etwas ändert.

Drei Blicke auf dieselben Daten

Die drei beschreibenden Dateien klingen ähnlich, stellen aber verschiedene Fragen. Man kann sie sich wie drei Stationen beim Kochen vorstellen: die Kontrolle der angelieferten Zutaten, das Protokoll über das fertige Gericht und das Etikett, auf dem steht, was drin ist.

DIE EINGANGSKONTROLLE
rohbefunde.py
Was kam vom Portal?
Blick auf: die Rohdaten
Beispiel: Wie viele Meldungen sind doppelt?

DAS MESSPROTOKOLL
deskriptiv.py
Wie sieht die fertige Tabelle aus?
Blick auf: die beiden Analysedateien
Beispiel: Wie stark schwankt die Armutsquote zwischen den Stadtteilen?

DAS LEXIKON
codebook.py
Was bedeutet jede Spalte?
Blick auf: die beiden Analysedateien
Beispiel: Ist `armutsquote_pct` ein Anteil oder ein Prozentwert?

Abbildung 2: Drei Dateien, drei Fragen an dieselben Daten. Keine von ihnen verändert eine Datei der Aufbereitung, alle drei schreiben nur nach `results/`.

Beschreiben ist nicht Entscheiden

`deskriptiv.py` stellt zum Beispiel fest, dass die Einsatzzahlen viel stärker streuen, als ein einfaches Zählmodell erwarten lässt. Was daraus für die Wahl der Verfahren folgt, steht bewusst nicht hier, sondern in der Eignungsprüfung (Heft 2, `v2_eignung.py`). Deshalb darf `deskriptiv.py` auf allen 36 Stadtteilen rechnen, auch auf den sechs zurückgehaltenen: Wer nur beschreibt und nichts entscheidet, kann kein Ergebnis vorwegnehmen.

Zwei Kontrollen zu zwei Zeitpunkten

`s2_datensaetze.py` prüft schon beim Bauen, ob die Zahl der Stadtteile und die Einwohnerzahl der Stadt stimmen, und bricht sonst ab. Die Prüfdatei in `tests/` kontrolliert danach das fertige Produkt. Sie prüft die Dateien selbst und ruft nur für einige Gegenrechnungen Funktionen aus Schritt 2 auf. Damit fiele auch auf, wenn jemand eine Tabelle von Hand verändert hätte.

Zwei Einstellungsdateien

`config.py` legt fest, was in die Dateien geschrieben wird: Zeitraum, Merkmale, Einsatzarten, Zahl der Prüfgruppen. Was nur beim Rechnen gilt, etwa die Suchräume der Verfahren, steht in

`modelle/config_modelle.py` (Heft 3). So gibt es keine zwei Dateien, die zwingend übereinstimmen müssen.

DREI SPERREN GEGEN UNERLAUBTES VORWISSEN

Ein Vorhersagemodell darf nur mit Informationen rechnen, die es im Ernstfall auch hätte (Band 2, Teil 6). In diesem Ordner sitzt jede dieser Sperren an genau einer Stelle, und jede ist eine eigene Funktion.

Erstens nimmt `acs_snapshot()` in `s1_daten.py` nur Sozialdaten, die damals schon veröffentlicht waren. **Zweitens** zählt `kriminalitaetsindex()` nur Straftaten der zwölf Vormonate. Beide schützen vor einem Blick in die Zukunft. **Drittens** sorgt `ergaenze_aufteilung()` in `s2_datensaetze.py` dafür, dass kein Stadtteil zugleich zum Üben und zum Prüfen dient. Das schützt vor Vorwissen über den geprüften Stadtteil selbst.

TEIL 3

Datei für Datei

> Für jede Datei: was sie tut, was sie liest und schreibt, und eine Tabelle mit jeder Funktion und ihrem Zweck. Die Reihenfolge folgt dem Ablauf, nicht dem Alphabet.

SO LIEST MAN DIE TABELLEN

Eine Python-Datei ist eine Art Rezept. Eine *Funktion* ist ein einzelner, benannter Arbeitsschritt darin: Sie bekommt etwas hinein, tut etwas damit und gibt etwas zurück. Funktionen, deren Name mit einem Unterstrich beginnt (etwa `_get`), sind als interne Hilfsschritte gedacht. Einige davon leihen sich andere Dateien trotzdem aus. Die Funktion `main` oder `run` ist der *Startpunkt*: Sie läuft, wenn man die Datei aufruft, und ruft ihrerseits die übrigen Funktionen in der richtigen Reihenfolge auf.

In den folgenden Tabellen steht jede Funktion einzeln, in der Reihenfolge, in der sie im Quelltext vorkommt. Die Markierung *Startpunkt* zeigt, wo der Ablauf beginnt, *Hilfsfunktion* einen kleinen Zuarbeiter, *innen* eine Funktion, die nur innerhalb einer anderen Funktion existiert. Die Funktionszahlen in den Übersichten zählen solche inneren Funktionen mit.

---

prep/

config.py

Das Regelbuch

Alle Festlegungen der Aufbereitung an einer Stelle.

LIEST	SCHREIBT	BENUTZT VON
nichts	nichts	fast allen anderen Dateien des Projekts

`config.py` enthält keine einzige Funktion, nur festgelegte Werte. Das ist Absicht: Wer eine Festlegung ändern will, ändert sie hier und nur hier, und alle anderen Dateien lesen sie von dort.

Darin stehen unter anderem die Speicherorte, die Download-Schalter (alle ausgeschaltet, gerechnet wird aus den gespeicherten Rohdaten), die Jahrgänge der Sozialdaten, der Publikationsversatz von einem Jahr, das Zwölf-Monats-Fenster des Kriminalitätsindex, der Zeitraum von Januar 2015 bis Dezember 2025, die drei ausgeschlossenen Parkgebiete, die zehn Merkmale samt den beiden Saisonwerten, die vier Einsatzarten, die Liste der Spalten, die erst nach einem Einsatz feststehen und deshalb nie Merkmal werden dürfen, und die Zahl der Prüfgruppen: fünf.

Eine Festlegung verdient Erwähnung, weil sie leicht missverstanden wird: Die Werte der Vormonate stehen zwar in der Tabelle, sind aber ausdrücklich kein Merkmal. Sonst erklärte die Vergangenheit das Ergebnis statt der Struktur des Stadtteils.

Keine Funktionen, nur Festlegungen.

prep/

build.py

Der eine Befehl

Startet die ganze Aufbereitung mit `python prep/build.py`.

LIEST	SCHREIBT
über Schritt 1 und 2 die Rohdaten in `data/raw/`	`data/processed/regression.parquet` und `klassifikation.parquet`

`build.py` rechnet selbst nichts. Es ruft Schritt 1 und Schritt 2 in der vorgeschriebenen Reihenfolge auf, misst die Dauer und zeigt am Ende, was entstanden ist. Mit dem Zusatz `tests` laufen danach gleich die Prüfungen.

FUNKTION	WOZU SIE DA IST
`schritt()` Hilfsfunktion	Schreibt die Überschrift eines Arbeitsschritts auf den Bildschirm und startet die Stoppuhr für diesen Schritt.
`uebersicht()`	Zeigt einen Steckbrief der erzeugten Dateien: Zeilen, Spalten, Dateigröße, Zeitraum. Fehlt eine Datei, steht dort FEHLT, statt dass der Lauf abbricht.
`main()` Startpunkt	Startet Schritt 1 und Schritt 2 nacheinander und danach die Übersicht. Die Reihenfolge ist zwingend, weil Schritt 2 liest, was Schritt 1 schreibt.

prep/

s1_daten.py

Schritt 1: Rohdaten holen und an jeden Einsatz hängen

Führt alle Quellen auf der Ebene des einzelnen Einsatzes zusammen.

LIEST	SCHREIBT	BENUTZT VON
`data/raw/`, aus dem Netz nur bei eingeschaltetem Download-Schalter	`data/processed/einsaetze.parquet` und Zwischendateien	rohbefunde (`tract_zu_stadtteil`) und panelprofil (`neighborhoods_gdf`)

Diese Datei ist der aufwendigste Teil der Aufbereitung. Sie arbeitet bewusst noch nicht mit Stadtteil-Monaten, sondern mit jedem einzelnen Einsatz. Der Grund: Nur ein einzelner Einsatz hat einen eigenen Zeitpunkt, und nur an diesem Zeitpunkt lässt sich entscheiden, welche Sozialdaten damals schon veröffentlicht waren. Verdichtet wird erst in Schritt 2.

Die Funktionen folgen der Reihenfolge der Arbeit: erst beschaffen, dann die Feuerwehrdaten aufräumen, dann Sozialdaten, Kriminalität und Bebauung zuordnen, zuletzt alles zusammenführen.

FUNKTION	WOZU SIE DA IST
BESCHAFFEN	
`_get()` Hilfsfunktion	Ruft eine Internetadresse ab und meldet einen Fehler, wenn die Antwort nicht stimmt. Hilfsschritt für die Downloads von DataSF.

FUNKTION	WOZU SIE DA IST
`lade_datasf()`	Holt einen kompletten Bestand vom Datenportal der Stadt (DataSF), Seite für Seite, weil das Portal je Anfrage nur eine begrenzte Zahl Zeilen liefert. Legt die Spaltentypen fest, damit etwa führende Nullen in Bezirksnummern nicht verloren gehen.
`lade_acs()`	Holt die Sozialdaten des amerikanischen Statistikamts (American Community Survey) für einen Jahrgang, aufgeteilt nach kleinen Zensusbezirken. Eigene Funktion, weil das Statistikamt ein anderes Format liefert als das Stadtportal.
`run_download()` Startpunkt	Lädt jede Quelle, deren Download-Schalter eingeschaltet ist, neu nach `data/raw/` und überschreibt dabei den gespeicherten Stand. Stehen alle Schalter auf aus, passiert hier nichts, und gerechnet wird mit dem gespeicherten Stand.
FEUERWEHRDATEN AUFRÄUMEN	
`prepare_sffd()`	Räumt die Einsatzmeldungen auf: entfernt doppelte Meldungen desselben Einsatzes, berechnet die Antwortzeit und behält nur Einsätze mit 0 bis 60 Minuten, ergänzt Jahr, Monat und Uhrzeit und vereinheitlicht die Schreibweise der Stadtteilnamen.
SOZIALDATEN ZUORDNEN	
`tract_zu_stadtteil()`	Übersetzt Zensusbezirke in Stadtteile. Weil das Statistikamt Bezirke von Zeit zu Zeit teilt, greift die Funktion notfalls auf die ersten vier Ziffern der Bezirksnummer zurück, die bei einer Teilung gleich bleiben (Band 2, Teil 6).
`acs_je_neighborhood()`	Fasst die Werte der Zensusbezirke zu Stadtteilwerten zusammen: Zählgrößen werden addiert, Medianwerte nach Einwohnern gewichtet gemittelt. Bricht ab, wenn in einem Jahrgang zu wenige Einwohner zugeordnet werden konnten.
`acs_snapshot()`	Entscheidet für ein Einsatzjahr, welcher Jahrgang der Sozialdaten damals schon veröffentlicht war: höchstens Einsatzjahr minus eins. Die erste der drei Sperren gegen unerlaubtes Vorwissen.
`join_acs()`	Hängt an jeden Einsatz die Sozialdaten seines Stadtteils aus dem Jahrgang, den `acs_snapshot()` bestimmt hat.
KRIMINALITÄT UND BEBAUUNG ZUORDNEN	
`neighborhoods_gdf()`	Lädt die Stadtteilgrenzen als Flächen. Kriminalität und Bebauung werden über dieselben Flächen zugeordnet, damit sich beide auf genau dieselben Gebiete beziehen.
`crime_monatlich()`	Zählt die Straftaten je Stadtteil und Monat aus den beiden Polizeibeständen, dem älteren bis Ende 2017 und dem neueren ab 2018. Im älteren fehlt der Stadtteil, dort wird jede Tat über ihre Koordinaten einer Fläche zugeordnet.

FUNKTION	WOZU SIE DA IST
`kriminalitaetsindex()`	Macht aus den Straftaten den relativen Kriminalitätsindex: die Belastung eines Stadtteils in den zwölf Vormonaten im Verhältnis zur ganzen Stadt. 1,0 heißt „so belastet wie der Stadtdurchschnitt“. Weil das Fenster im Vormonat endet, ist das die zweite Sperre.
`land_use_je_neighborhood()`	Ordnet jedes Grundstück über seinen Mittelpunkt einem Stadtteil zu und zählt je Stadtteil, was die baulichen Merkmale brauchen: Grundstücke, Altbauten, Wohngrundstücke und brandlastreiche Gewerbefläche. Die Anteile daraus rechnet später `berechne_quoten()`. Dafür gibt es nur einen Stand, das Jahr 2020.
ZUSAMMENFÜHREN	
`berechne_quoten()`	Rechnet Anteile zwischen 0 und 1 aus, etwa die Armutsquote. Ist der Nenner null, bleibt das Feld leer, statt dass durch null geteilt wird.
`run_join()` Startpunkt	Führt alles zur Einsatztabelle zusammen: Feuerwehrdaten aufräumen, Sozialdaten anhängen, Kriminalitätsindex und Bebauung über die Stadtteilflächen dazuspielen, Quoten rechnen, Datei schreiben. Mit `python prep/s1_daten.py join` läuft nur dieser Teil, ohne Download.

prep/

s2_datensaetze.py

Schritt 2: Von Einsätzen zu Stadtteil-Monaten

Baut die beiden Analysetabellen und legt die Prüfgruppen fest.

LIEST	SCHREIBT	BENUTZT VON
`data/processed/einsaetze.parquet`	`regression.parquet` (4.752 Zeilen) und `klassifikation.parquet` (4.751 Zeilen)	fast allen späteren Skripten, über die Funktion `fold_masken()`, sowie build und den Tests

Hier entsteht die Einheit, mit der die ganze Arbeit rechnet: ein Stadtteil in einem Monat. Die Datei hat drei Teile. Teil A legt fest, welcher Stadtteil in welcher Prüfgruppe steht. Teil B baut die Tabelle für die Frage „Wie viele?“, Teil C die für „Welche Art?“.

Die Gruppenzuteilung wird als Spalte in beide Dateien geschrieben. Dass alle Verfahren dieselben Gruppen sehen, ist damit eine Eigenschaft der Daten und nicht nur ein Versprechen des Codes. Der Klassifikationstabelle fehlt genau eine Zeile: der eine Monat ohne Einsatz, dem sich keine Einsatzart zuordnen lässt.

FUNKTION	WOZU SIE DA IST
TEIL A · AUFTEILUNG	
ergaenze_aufteilung()	Verteilt die 36 Stadtteile reihum auf sechs Gruppen: Gruppe 0 ist das Hold-out, die Gruppen 1 bis 5 sind die Prüfgruppen. Vorher wird sortiert, nach der Zahl der Monate, in denen Brände überwiegen, und bei Gleichstand nach Einwohnern. So bekommt jede Gruppe Brandfälle und eine ähnliche Größenmischung. Schreibt die Spalten `fold` und `ist_holdout`. Die dritte Sperre: Kein Stadtteil ist zugleich Übungs- und Prüffall.
fold_masken()	Beantwortet für eine Prüfgruppe: Welche Zeilen dienen zum Üben, welche zum Prüfen? Geprüft wird ein Stadtteil immer mit allen seinen Monaten, das Hold-out ist nie dabei.
beschreibe_splits()	Zeigt die Aufteilung lesbar am Bildschirm: welcher Stadtteil in welcher Gruppe steht und dass jede Gruppe den ganzen Zeitraum abdeckt. Läuft mit `python prep/s2_datensaetze.py splits`.
TEIL B · MENGE	
_monat_minus() Hilfsfunktion	Rechnet einen Monatsschlüssel wie 202403 um eine bestimmte Zahl von Monaten zurück.
_setze_datentypen() Hilfsfunktion	Bringt alle Spalten auf einheitliche Zahlentypen. Nötig, weil XGBoost eine Spalte mit einem pandas-eigenen Zahlentyp ablehnt, während die anderen Verfahren ihn still umwandeln.
aggregiere()	Verdichtet die Einsätze zu einer Zeile je Stadtteil und Monat. Auch ein Monat ohne Einsatz bekommt eine Zeile mit dem Wert null, damit das Raster lückenlos bleibt.
baue_regression()	Baut die Tabelle für „Wie viele?“: die Merkmale, beide Zielgrößen (Zahl der Einsätze und Einsätze je 1.000 Einwohner), die Einwohnerzahl, die Saison und zur Beschreibung die Werte der Vormonate. Dafür werden zwölf Monate vor 2015 mitgerechnet und danach wieder abgeschnitten.
TEIL C · STRUKTUR	
baue_klassifikation()	Baut die Tabelle für „Welche Art?“: die Anteile der vier Einsatzgruppen je Stadtteil-Monat und daraus die überwiegende Einsatzart. Zeilen und Merkmale übernimmt sie aus der Regressionstabelle, damit beide Stränge auf derselben Grundlage stehen. Die Gruppenzuteilung trägt `run()` danach in beide Tabellen gleich ein.
ABSCHLUSS	
pruefe_zuschnitt()	Prüft zwei Zahlen, die ein stiller Datenverlust zwangsläufig verändern würde: die Zahl der Stadtteile und die Einwohnerzahl der ganzen Stadt. Passt eine nicht, bricht der Lauf ab.

FUNKTION	WOZU SIE DA IST
run() Startpunkt	Baut beide Tabellen, teilt die Gruppen einmal zu, trägt sie in beide ein und schreibt die Dateien.

prep/

rohbefunde.py

Die Eingangskontrolle

Beschreibt die Rohdaten so, wie sie vom Portal kommen.

LIEST
die Rohdaten in `data/raw/`, dazu die Sozialdaten je Stadtteil aus Schritt 1

SCHREIBT
`results/deskriptiv/rohbefunde.md` und `rohbefunde_acs.csv`

Diese Datei hält fest, wie die Lieferung aussah. Sie läuft nach der Aufbereitung, weil sie für die Sozialdaten auch auf deren Zuordnung zu den Stadtteilen zurückgreift. Aufgenommen ist nur, was in der Aufbereitung eine Folge hat, zum Beispiel: wie viele Meldungen es gibt und wie viele davon im Analysezeitraum liegen, wie viele doppelt sind, wie viele Grundstücke kein Baujahr haben, wie viele Zensusbezirke jedes Jahrgangs direkt und wie viele erst über die ersten vier Ziffern ihrer Nummer einem Stadtteil zugeordnet werden und wie wenige Menschen in den Parkgebieten wohnen. Zu fast jedem Befund gehört ein Eingriff in Schritt 1 oder Schritt 2, bei Mission Bay ist keiner nötig. Für die Zuordnung der Zensusbezirke benutzt die Datei dieselbe Funktion wie die Aufbereitung, `tract_zu_stadtteil()` aus Schritt 1.

FUNKTION	WOZU SIE DA IST
z() Hilfsfunktion	Schreibt Zahlen in deutscher Form: Punkt als Tausender-, Komma als Dezimaltrenner.
main() Startpunkt	Liest die Rohquellen, ermittelt die Befunde und schreibt sie als kurzen Bericht und als Tabelle zu den Sozialdaten-Jahrgängen.

prep/

deskriptiv.py

Das Messprotokoll

Beschreibt, wie die fertige Tabelle aussieht.

LIEST
beide Analysedateien, alle 36 Stadtteile

SCHREIBT
`results/deskriptiv/`: sieben Tabellen und die Lesefassung `befunde.md`

`deskriptiv.py` misst Lage, Streuung und Form der Merkmale und Zielgrößen und vergleicht zwei Zusammenhangsmaße. Ihr Kern ist die Varianzzerlegung: Bei den meisten Merkmalen liegt der Großteil der Schwankung zwischen den Stadtteilen und nur wenig innerhalb eines Stadtteils über die Zeit; bei Miete und Einkommen sind es nur gut die Hälfte und zwei Drittel, die Saisonterme schwanken allein über die Zeit. Daraus folgt die Rechnung aus Band 2, Teil 7: Die vielen Zeilen tragen weit weniger unabhängige Information, als ihre Zahl vermuten lässt.

Die Lesefassung `befunde.md` nennt gleich oben, auf welcher Menge gerechnet ist: alle 36 Stadtteile, Entwicklung und Hold-out.

FUNKTION	WOZU SIE DA IST
`gruppe_von()` Hilfsfunktion	Sagt, zu welcher Merkmalsgruppe eine Spalte gehört: sozioökonomisch, kriminalitätsbezogen, baulich, Größenkontrolle oder Saison.
`verteilung()`	Lage, Streuung und Form je Merkmal: Mittelwert, Median, Streuung, Schiefe und Wölbung. Beschreibt nur, ein formaler Test auf Normalverteilung steht in der Eignungsprüfung.
`varianzzerlegung()`	Misst je Merkmal, welcher Teil der Schwankung zwischen den Stadtteilen und welcher innerhalb eines Stadtteils über die Zeit liegt. Daraus folgen Intraklassenkorrelation, Designeffekt und effektive Stichprobe.
`aufloesung()`	Zählt, wie viele verschiedene Werte ein Merkmal je Stadtteil annimmt. Ein einziger Wert heißt: Das Merkmal steht 132-mal gleich in der Tabelle und bringt über die Zeit keine neue Information.
`stadtteilprofil()`	Eine Zeile je Stadtteil mit der typischen Einsatzlast über alle Monate, der Einwohnerzahl und der Kennzeichnung, ob er zum Hold-out gehört.
`zielgroessen()`	Steckbrief der beiden Zielgrößen. Wichtigster Wert ist der Dispersionsindex: Er zeigt, wie viel stärker die Einsatzzahlen streuen, als es bei einer Poisson-Verteilung zu erwarten wäre. Dazu kommen der Anteil der Monate ohne Einsatz und die Einwohnerzahl der Stadt. Für die Einsatzart zeigt die Funktion außerdem, wie oft zwei Einsatzgruppen gleichauf liegen und dass sich die Frage je Einsatz nicht, je Stadtteil-Monat aber sehr wohl beantworten lässt.
`zusammenhaenge()`	Berechnet je Merkmal zwei Zusammenhangsmaße mit der Zielgröße, Pearson (gerade Linie) und Spearman (gleiche Richtung), und ihren Abstand. Ein großer Abstand deutet auf einen gekrümmten Zusammenhang. Liegen beide nahe null, wird der Abstand als nicht belastbar markiert.
`z()` Hilfsfunktion	Schreibt Zahlen in deutscher Form, ohne dabei die Kommas im Fließtext anzutasten.
`als_markdown()` Hilfsfunktion	Wandelt eine Tabelle in eine lesbare Texttabelle (Markdown) mit deutschen Zahlen um.
`main()` Startpunkt	Liest beide Dateien, ruft alle Auswertungen auf und schreibt die Tabellen und die Lesefassung `befunde.md`. Mit `-v` kommen die Rohwerte je Stadtteil dazu.

prep/

codebook.py

Das Lexikon der Merkmale

Erklärt jede Spalte: Skalenniveau, Einheit, Quelle, Bedeutung.

LIEST	SCHREIBT
beide Analysedateien	`results/codebook/merkmale.csv` und `merkmale.md`

Das Codebook ist eine einzige große Tabelle über alle Spalten. Eine Hälfte davon wird bei jedem Lauf neu gemessen, etwa der Wertebereich und die Zahl fehlender Werte. Sie kann deshalb nicht veralten. Die andere Hälfte, also Skalenniveau, Einheit, Quelle und die Erklärung, steht so in keiner Datei und wird von Hand gepflegt. Ein Wächter sorgt dafür, dass beide Hälften zusammenpassen: Kommt eine Spalte hinzu oder fällt eine weg, bricht das Skript ab, bis das Lexikon nachgezogen ist.

EIN BEFUND, DER IN DIE TABELLE GEHÖRT

Sechs Spalten tragen die Endung `_pct`, enthalten aber Anteile von 0 bis 1 und keine Prozentwerte: `armutsquote_pct` steht auf 0,36 und meint 36 %. Umbenannt wird nichts, weil die Namen in allen Ergebnissen stehen. Das Codebook weist es stattdessen in der Spalte Einheit aus.

FUNKTION	WOZU SIE DA IST
`Meta` Bauplan (Klasse)	Kein Arbeitsschritt, sondern ein Bauplan: Er legt fest, welche Angaben je Merkmal von Hand gepflegt werden (Skalenniveau, Einheit, Quelle, Was, Wie, Wofür) und ob eine Zahl eine Kennung ist, etwa eine Jahreszahl, die keinen Tausenderpunkt bekommen darf.
`M()` Hilfsfunktion	Kurzschreibweise, um einen solchen Eintrag anzulegen. Damit bleibt die lange Liste der Einträge übersichtlich.
`_de()` Hilfsfunktion	Deutsche Zahlformatierung mit Tausenderpunkt.
`spanne()`	Bestimmt den Wertebereich einer Spalte: bei Zahlen „kleinster bis größter Wert“, sonst die vorkommenden Ausprägungen.
`gemessen()`	Misst je Spalte den gemessenen Teil: Wertebereich, Zeilenzahl, fehlende Werte, Zahl der Ausprägungen und in welcher der beiden Dateien sie vorkommt.
`waechter()`	Vergleicht Daten und Lexikon. Steht eine Spalte in den Daten, aber nicht im Lexikon, oder ein Eintrag im Lexikon ohne passende Spalte, meldet er das. Dann bricht das Skript ab.
`baue()`	Lässt erst den Wächter laufen und setzt dann gemessenen und gepflegten Teil zu einer Tabelle zusammen.

FUNKTION	WOZU SIE DA IST
als_markdown() Hilfsfunktion	Schreibt die Tabelle als Lesefassung, mit zwei Sätzen Lesehilfe davor.
main() Startpunkt	Prüft, ob die beiden Analysedateien vorhanden sind, baut die Tabelle und schreibt sie als CSV und als Lesefassung.

tests/

test_aufbereitung.py

Die Endkontrolle

Prüft die beiden fertigen Tabellen mit 20 Prüfungen.

LIEST
beide Analysedateien und `einsaetze.parquet`

SCHREIBT
nichts, meldet je Prüfung OK oder FEHLER

Die Prüfdatei ist bewusst vom übrigen Code getrennt. Sie schaut nicht nach, wie die Tabellen gebaut wurden, sondern prüft das Ergebnis selbst; nur für einige Gegenrechnungen leiht sie sich Funktionen aus Schritt 2. Jede Prüfung ist eine eigene Funktion, deren Name mit `test_` beginnt. Die 20 Prüfungen fallen in vier Gruppen und eine Ergänzung: Ist die Tabelle vollständig, ist die Aufteilung sauber, sind die Merkmale frei von Zukunftswissen, passt der Strukturteil zum Mengenteil, und sind die Werte selbst plausibel?

FUNKTION	WOZU SIE DA IST
LADEN	
regression() Hilfsfunktion	Liest die Regressionstabelle einmal ein und hält sie für alle Prüfungen bereit. Fehlt die Datei, kommt der Hinweis, zuerst `build.py` zu starten.
klassifikation() Hilfsfunktion	Dasselbe für die Klassifikationstabelle.
1 · DIE TABELLE	
test_panel_rechteckig_und_vollstaendig()	Jeder Stadtteil hat jeden Monat genau einmal, und kein Merkmalswert fehlt.
test_zeitraum_festgesetzt()	Der Zeitraum stammt aus den Festlegungen in `config.py`, nicht aus dem, was zufällig in den Daten steht. Sonst verschöbe ihn jeder neue Download.
test_datentypen_modelltauglich()	Keine Spalte hat einen Zahlentyp, an dem XGBoost scheitern würde.
test_exposure_und_kriminalitaetsindex_vorhanden()	Die logarithmierte Einwohnerzahl und der logarithmierte Kriminalitätsindex sind gebildet und vollständig.
2 · DIE AUFTEILUNG	
test_folds_ordnung_und_holdout()	Kein Stadtteil ist zugleich Übungs- und Prüffall, und das Hold-out bleibt getrennt.

FUNKTION	WOZU SIE DA IST
test_jeder_fold_deckt_den_vollen_zeitraum()	Ein Prüfstadtteil wird mit allen seinen Monaten geprüft, nicht nur mit einem Teil davon.
test_aufteilungsspalten_konsistent()	Die Gruppenspalten in der Datei passen zur Zuteilungsregel in `s2_datensaetze.py`. Das fällt auf, wenn die Regel geändert, die Datei aber nicht neu gebaut wurde.
test_folds_decken_die_groessenspanne_ab()	Keine Prüfgruppe besteht nur aus großen Stadtteilen.
3 · DIE MERKMALE	
test_merkmale_vollstaendig()	Alle Merkmale sind vorhanden, und die Rate je 1.000 Einwohner ist gebildet.
test_saison_zyklisch()	Die beiden Saisonwerte liegen auf einem Kreis, damit Dezember an Januar grenzt, statt elf Monate von ihm entfernt zu sein.
test_lags_gegen_rohdaten()	Schlägt Vormonatswerte stichprobenartig direkt in den Einsatzdaten nach. Ein um einen Monat verrutschter Wert wäre im Code nicht zu sehen, hier fällt er auf.
test_lags_nicht_gegenwartsbezogen()	Gegenprobe: Der Wert „Vormonat“ darf nicht der Wert des laufenden Monats sein.
test_vorlauf_ohne_eigene_zeilen()	Die zwölf Monate vor 2015 liefern nur Vormonatswerte, aber keine eigenen Zeilen.
4 · DER STRUKTURTEIL	
test_keine_ergebnisvariablen()	Keine Spalte, die erst nach dem Einsatz feststeht, etwa Sachschaden, Zahl der Fahrzeuge oder Antwortzeit, steckt in den Merkmalen. Die wichtigste Prüfung des Strukturteils.
test_struktur_gleiche_abgrenzung_wie_regression()	Beide Tabellen haben dieselben Stadtteile, denselben Zeitraum und dieselbe Gruppenzuteilung.
test_anteile_konsistent()	Die vier Anteile ergeben je Zeile zusammen 1 und passen zu den gezählten Einsätzen.
test_zielklasse_konsistent()	Die überwiegende Einsatzart ist tatsächlich die mit dem größten Anteil und kein künstlich gesetzter Schwellenwert.
test_seltene_klasse_in_jedem_fold()	Jede Prüfgruppe enthält Monate, in denen Brände überwiegen. Sonst fehlte einer Gruppe eine ganze Einsatzart.
test_struktur_hat_signal()	Die Anteile unterscheiden sich zwischen den Stadtteilen. Sonst gäbe es für die Merkmale nichts zu erklären.

FUNKTION	WOZU SIE DA IST
ERGÄNZUNG · DIE WERTE	
`test_exposition_plausibel()`	Die Einwohnerzahl der ganzen Stadt liegt in einer plausiblen Spanne, und kein Stadtteil wächst zwischen zwei Jahren um mehr als 60 % oder schrumpft um mehr als gut ein Drittel. Sichert die Werte, wo die übrigen Prüfungen die Form sichern.
START	
`main()` Startpunkt	Führt alle 20 Prüfungen nacheinander aus, meldet je Prüfung OK oder FEHLER und am Ende die Bilanz.

TEIL 4

In drei Sätzen

> Was man über diesen Ordner wissen muss, wenn man nur eine Minute hat.

1. In `prep/` erzeugen nur zwei Dateien Daten: `s1_daten.py` arbeitet auf dem einzelnen Einsatz, `s2_datensaetze.py` auf dem Stadtteil-Monat, `build.py` startet beide.

2. Drei Dateien sehen sich die Daten nur an: `rohbefunde.py` die Lieferung, `deskriptiv.py` die fertige Tabelle, `codebook.py` die Bedeutung jeder Spalte.

3. `tests/` prüft das fertige Produkt mit 20 Prüfungen, unabhängig davon, wie es entstanden ist.

Weiter geht es in Heft 2 mit dem Ordner `vorpruefung/`: Dort wird festgelegt, was ein Modell mindestens leisten muss.